

Question No:7

DS

Session

Tree 1

Question Information

Level 1 Challenge 67

Question description

The sam is enjoying a wonderful vacation in Byteland. What makes the sam impressed the most is the road system of the country. Byteland has N cities numbered 1 through N . City 1 is the capital of Byteland. The country also has $N-1$ bidirectional roads connecting the cities. The i -th road connects two different cities u_i and v_i . In this road system, people can travel between every pair of different cities by going through exactly one path of roads.

The roads are arranged in such a way that people can distinguish two cities only when both cities have different number of roads connected to it. Such two cities will be considered similar. For example, city A is similar to the capital if the number of roads connected to city A is equal to the number of roads connected to the capital.

On each day during the vacation, the sam wants to have a trip as follows. He chooses two cities A and B such that the sam will visit city B if he goes from A to the capital using the shortest path. Then, the sam will visit the cities on the shortest path from A to B through this path. Please note that A may be equal to B; that means the sam will enjoy the day in a single city.

The sam does not want to have similar trips. Two trips are considered similar if and only if they both have the same number of visited cities and for each i , the i -th city visited in one trip is similar to the i -th city visited in the other trip.

The sam wants to have as many different, namely not similar, trips as possible. Help him count the maximum number of possible trips such that no two of them are similar.

Input Format

The first line of the input contains a single integer N . The i -th line of next $N-1$ lines contains two space-separated integers u_i and v_i , denoting the i -th road.

Output Format

Output a single line containing the maximum number of different trips.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3
2 1
3 1

EXPECTED OUTPUT

3

Test Case 2

INPUT (STDIN)

4
2 1
3 1
2 4

EXPECTED OUTPUT

5

▼ Mandatory Test Cases

Test Case 1

KEYWORD

struct state

Test Case 2

KEYWORD

const int MAXL=200005;

Test Case 3

KEYWORD

state st[MAXL]

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

700

Test Case 3

NLOC

67

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4
5  #define MAXN 100005
6  #define MAXS 200005
7  #define MAXT 600005
8  #define HASH_SIZE (1u << 21)
9
10 struct state {
11     int len;
12     int link;
13     int head;
14 };
15
16 struct transition {
17     int label;
18     int to;
19     int next;
20 };
21
22 struct state st[MAXS];
23 struct transition tr[MAXT];
24
25 uint64_t hkey[HASH_SIZE];
26 int hval[HASH_SIZE];
27
28 int sz;
29 int last;
```

```
30  int trCnt;
31
32  /* Hash function */
33  unsigned long long hash_key(uint64_t x) {
34      x ^= x >> 33;
35      x *= 0xff51afd7ed558ccdULL;
36      x ^= x >> 33;
37      x *= 0xc4ceb9fe1a85ec53ULL;
38      x ^= x >> 33;
39
40      return x;
41  }
42
43  /* Find hash position */
44  int find_slot(int from, int label) {
45
46      uint64_t key =
47          ((uint64_t)(unsigned int)from << 32)
48          | (unsigned int)label;
49
50      unsigned int p =
51          (unsigned int)hash_key(key)
52          & (HASH_SIZE - 1);
53
54      while (hkey[p] != 0 && hkey[p] != key)
55          p = (p + 1) & (HASH_SIZE - 1);
```

```

57     return (int)p;
58 }
59
60 /* Get transition */
61 int get_next(int from, int label) {
62
63     int p = find_slot(from, label);
64
65     if (hkey[p] == 0)
66         return -1;
67
68     return tr[hval[p]].to;
69 }
70
71 /* Set transition */
72 void set_next(int from, int label, int to) {
73
74     int p = find_slot(from, label);
75
76     if (hkey[p] != 0) {
77         tr[hval[p]].to = to;
78         return;
79     }
80
81     if (trCnt >= MAXT) {

```

```
82         printf("Too many transitions\n");
83         exit(1);
84     }
85
86     uint64_t key =
87         ((uint64_t)(unsigned int)from << 32)
88         | (unsigned int)label;
89
90     hkey[p] = key;
91     hval[p] = trCnt;
92
93     tr[trCnt].label = label;
94     tr[trCnt].to = to;
95     tr[trCnt].next = st[from].head;
96
97     st[from].head = trCnt;
98
99     trCnt++;
100 }
101
102 /* Initialize suffix automaton */
103 void sa_init() {
104
105     sz = 1;
106     last = 0;
107     trCnt = 0;
```

```
109     st[0].len = 0;
110     st[0].link = -1;
111     st[0].head = -1;
112 }
113
114 /* Copy transitions while cloning */
115 void copy_transitions(int from, int to) {
116
117     int e = st[from].head;
118
119     while (e != -1) {
120
121         set_next(
122             to,
123             tr[e].label,
124             tr[e].to
125         );
126
127         e = tr[e].next;
128     }
129 }
130
131 /* Extend suffix automaton */
132 void sa_extend(int c) {
133
134     int cur = sz++;
```

```
136     st[cur].len =
137         st[last].len + 1;
138
139     st[cur].link = 0;
140     st[cur].head = -1;
141
142     int p = last;
143
144     while (p != -1 &&
145           get_next(p, c) == -1) {
146
147         set_next(p, c, cur);
148
149         p = st[p].link;
150     }
151
152     if (p == -1) {
153
154         st[cur].link = 0;
155     }
156     else {
157
158         int q = get_next(p, c);
159
160         if (st[p].len + 1 == st[q].len) {
```

```
161
162     st[cur].link = q;
163 }
164 else {
165
166     int clone = sz++;
167
168     st[clone].len =
169         st[p].len + 1;
170
171     st[clone].link =
172         st[q].link;
173
174     st[clone].head = -1;
175
176     copy_transitions(q, clone);
177
178     while (p != -1 &&
179         get_next(p, c) == q) {
180
181         set_next(
182             p,
183             c,
184             clone
```

```
185         );
186
187         p = st[p].link;
188     }
189
190     st[q].link = clone;
191     st[cur].link = clone;
192 }
193 }
194
195     last = cur;
196 }
197
198 /* Tree */
199 struct edge {
200     int to;
201     int next;
202 };
203
204 struct edge graph[2 * MAXN];
205
206 int head[MAXN];
207 int degreeNode[MAXN];
```

```

209     int edgeCount = 0;
210
211     void add_edge(int u, int v) {
212         graph[edgeCount].to = v;
213         graph[edgeCount].next = head[u];
214         head[u] = edgeCount++;
215     }
216
217     /* DFS frame */
218     struct frame {
219         int u;
220         int parent;
221         int edge;
222         int restoreLast;
223     };
224
225     /* Build all root-to-node degree strings */
226     void build_strings(int n) {
227         struct frame *stack =
228             malloc((size_t)n * sizeof(struct frame));
229         int top = 0;

```

```
235     sa_extend(degreeNode[1]);
236
237     stack[0].u = 1;
238     stack[0].parent = 0;
239     stack[0].edge = head[1];
240     stack[0].restoreLast = 0;
241
242     while (top >= 0) {
243
244         struct frame *f = &stack[top];
245
246         if (f->edge == -1) {
247
248             last = f->restoreLast;
249
250             top--;
251
252             continue;
253         }
254
255         int e = f->edge;
256
257         f->edge = graph[e].next;
258
259         int v = graph[e].to;
```

```
260
261     if (v == f->parent)
262         continue;
263
264     int oldLast = last;
265
266     sa_extend(degreeNode[v]);
267
268     top++;
269
270     stack[top].u = v;
271     stack[top].parent = f->u;
272     stack[top].edge = head[v];
273     stack[top].restoreLast = oldLast;
274 }
275
276 free(stack);
277 }
278
279 int main() {
280
281     int n;
282
283     scanf("%d", &n);
```

```

285     for (int i = 1; i <= n; i++)
286         head[i] = -1;
287
288     for (int i = 0; i < n - 1; i++) {
289
290         int u, v;
291
292         scanf("%d %d", &u, &v);
293
294         add_edge(u, v);
295         add_edge(v, u);
296
297         degreeNode[u]++;
298         degreeNode[v]++;
299     }
300
301     sa_init();
302
303     build_strings(n);
304
305     /*
306      |   Counting sort states according to length.
307      */
308
309     int *cnt =
310         calloc((size_t)n + 1, sizeof(int));

```

```
311
312     int *order =
313         malloc((size_t)sz * sizeof(int));
314
315     long long *dp =
316         calloc((size_t)sz, sizeof(long long));
317
318     for (int i = 0; i < sz; i++)
319         cnt[st[i].len]++;
320
321     for (int i = 1; i <= n; i++)
322         cnt[i] += cnt[i - 1];
323
324     for (int i = sz - 1; i >= 0; i--)
325         order[--cnt[st[i].len]] = i;
326
327     /*
328      * Count paths in suffix automaton.
329      */
330
331     for (int i = sz - 1; i >= 0; i--) {
332
333         int u = order[i];
334
335         dp[u] = 1;
336
337         int e = st[u].head;
```

```

338
339     while (e != -1) {
340
341         dp[u] +=
342             dp[tr[e].to];
343
344         e = tr[e].next;
345     }
346 }
347
348 printf("%lld\n", dp[0] - 1);
349
350 free(cnt);
351 free(order);
352 free(dp);
353
354 return 0;
355 }

```

Output:

Test Case-1:

```

3
2 1
3 1
3

```

Test Case-2:

	4	
	2 1	
	3 1	
	2 4	
	5	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236